

main.rb

```
1 # 1)
2 require 'pry'
3
4 require 'tadb'
5
6 module Boolean
7 end
8
9 class TrueClass
10   include Boolean
11 end
12
13 class FalseClass
14   include Boolean
15 end
16
17 module Persistible
18   def save!
19     if @id.nil?
20       define_singleton_method('id') do
21         @id
22       end
23     end
24     @id = self.class.save!(self)
25   end
26
27   def refresh!
28     raise 'Falla! Este objeto no tiene id!' if @id.nil?
29
30     objeto_persistido = self.class.buscar_objeto(@id)
31     self.class.tipos.keys.each do |atributo|
32       send("#{atributo}=", objeto_persistido.send(atributo))
33     end
34   end
35
36   def forget!
37     raise 'Falla! Este objeto no tiene id!' if @id.nil?
38
39     self.class.forget!(id)
40     remove_instance_variable('@id')
41   end
42 end
43
44 class Tabla
45   def initialize(owner)
46     @owner = owner
47   end
```

```
48   unless @owner.instance_of?(Module)
49     @tabla = TADB::DB.table("#{owner}")
50     define_singleton_method('entradas') do
51       hashees = @tabla.entries
52
53       objetos_persisitidos = []
54
55       hashees.each { |hash| objetos_persisitidos.append(@owner.crear_segun_hash(hash)) }
56
57       objetos_persisitidos
58     end
59
60     define_singleton_method('buscar_objeto') do |id|
61       hash = obtener_hash_id(id)
62
63       @owner.crear_segun_hash(hash)
64     end
65   end
66
67   nil
68 end
69
70 def eliminar_tabla
71   @tabla.clear
72 end
73
74 def method_missing(nombre_metodo, *argumentos, &block)
75   if nombre_metodo.to_s.start_with?('find_by_')
76     lista_de_objetos = all_instances
77     lista_de_objetos.select do |objeto|
78       objeto.send(nombre_metodo.to_s.delete_prefix('find_by_')) == argumentos.first
79     end
80   else
81     super(nombre_metodo, *argumentos, &block)
82   end
83 end
84
85 def respond_to_missing?(nombre_metodo, _include_private = false)
86   nombre_metodo.to_s.start_with?('find_by_') || super(nombre_metodo, false)
87 end
88
89 def all_instances
90   clases_de_las_tablas = @owner.hijos
91   clases_de_las_tablas.push(@owner) unless @tabla.nil?
92
93   lista_de_objetos = []
94
95   clases_de_las_tablas.each { |clase| clase.entradas.each { |entrada|
lista_de_objetos.push(entrada) } }
```

```
96
97     lista_de_objetos
98 end
99
100 def forget!(id)
101     # Debería eliminar también las tablas hijas?
102     @tabla.delete(id)
103 end
104
105 def obtener_hash_id(id)
106     @tabla.entries.find { |hash| hash[:id] == id }
107 end
108
109 def crear_entrada_sin_array(objeto, lista_atributos)
110     entrada = {}
111
112     lista_atributos.each do |atributo|
113         if objeto.send(atributo).respond_to?(:save!)
114             id = objeto.send(atributo).save!
115             entrada[atributo] = id
116         elsif !objeto.send(atributo).nil?
117             entrada[atributo] = objeto.send("#{atributo}")
118         end
119     end
120
121     unless objeto.id.nil?
122         entrada[:id] = objeto.id
123         forget!(objeto.id)
124     end
125
126     entrada
127 end
128
129 def crear_entrada_array(array)
130     conjunto_ids = []
131     array.each { |elemento| conjunto_ids.push(elemento.save!) }
132     conjunto_ids.join(',') # String de ids separados c/u por una coma
133 end
134
135 def save!(objeto)
136     if @owner.tipos.keys.any? { |atributo| objeto.send(atributo).is_a?(Array) }
137         atributos_listas = objeto.class.tipos.keys.select { |atributo|
objeto.send(atributo).is_a?(Array) }
138         atributos_comunes = objeto.class.tipos.keys.select { |atributo| !
objeto.send(atributo).is_a?(Array) }
139         hash = crear_entrada_sin_array(objeto, atributos_comunes)
140         atributos_listas.each { |atributo| hash[atributo] =
crear_entrada_array(objeto.send(atributo)) }
141         @tabla.insert(hash)
```

```
142     else
143       @tabla.insert(crear_entrada_sin_array(objeto, objeto.class.tipos.keys))
144     end
145   end
146 end
147
148 class Validador
149   def validar_default(hash, objeto)
150     return unless objeto.send(hash[:named]).nil?
151
152     objeto.send("#{hash[:named]}=", hash[:default])
153   end
154
155   def validate!(valor, tipo)
156     unless valor.is_a?(tipo) or (valor.is_a?(Array) and valor.all? { |e| e.is_a?(tipo) })
157 or valor.nil?
158       p valor
159       raise "Falla! el atributo no es de tipo #{tipo}!"
160     end
161
162     nil
163   end
164 end
165
166 class No_blank
167   def validar(key_value, atributo_valor)
168     if key_value and (atributo_valor.nil? or (atributo_valor.respond_to?(:empty?) and
169 atributo_valor.empty?))
170       raise 'Falla! el atributo no puede estar vacío!'
171     end
172
173     nil
174   end
175 end
176
177 class From
178   def validar(minimo, valor)
179     raise "Falla! #{valor} es menor que #{minimo}!" if minimo > valor and !valor.nil?
180
181     nil
182   end
183 end
184
185 class To
186   def validar(maximo, valor)
187     raise "Falla! #{valor} es mayor que #{maximo}!" if maximo > valor and !valor.nil?
188
189     nil
190   end
191 end
```

```
189 end
190
191 class Validate
192   def validar(block, valor)
193     if valor.is_a?(Array) and valor.any? { |elemento| !block.call(elemento) }
194       raise "Falla! existen elementos de la lista, que no cumplen con #{block}!"
195
196     elsif !block.call(valor)
197       raise "Falla! el atributo no cumple con #{block}!"
198     end
199
200     nil
201   end
202 end
203
204 class Module
205   def has_one(tipo, descripcion)
206     attr_accessor descripcion[:named]
207
208     if @tipos.nil?
209
210       if respond_to?(:new)
211         define_singleton_method('new') do |*args, &block|
212           objeto = super(*args, &block)
213
214           @listas_many.each { |elemento| objeto.send("#{elemento}=", []) } unless
215             @listas_many.nil?
216
217           validador = Validador.new
218           @tipos.keys.each do |atributo|
219             unless objeto.class.hashees_atributos[atributo][:default].nil?
220               validador.validar_default(objeto.class.hashees_atributos[atributo],
221                                     objeto)
222             end
223           end
224           objeto
225         end
226       end
227
228       define_singleton_method('save!') do |objeto|
229         validador = Validador.new
230         @tipos.keys.each do |atributo|
231           unless @hashees_atributos[atributo][:default].nil?
232             validador.validar_default(@hashees_atributos[atributo],
233                                   objeto)
234           end
235         end
236         validador.validate!(objeto.send(atributo), @tipos[atributo])
237         @hashees_atributos[atributo].except(:default, :named).each do |campo_extra|
```

```
236         # validador.send("validar_#{campo_extra}".to_sym, @hashees_atributos[atributo]
[campo_extra],
237         #             objeto.send(atributo))
238         Object.const_get(campo_extra.to_s.capitalize).new
239         .validar(@hashees_atributos[atributo][campo_extra],
objeto.send(atributo))
240     end
241 end
242 @tabla.save!(objeto)
243 end
244
245 forma_de_herencia = 'included'
246 @tabla = Tabla.new(self)
247 forma_de_herencia = 'inherited' unless instance_of?(Module)
248 @hijos = []
249
250 define_singleton_method('hijos') do
251     hijos_totales = []
252     @hijos.each { |hijo| hijos_totales.push(hijo) }
253     @hijos.each do |hijo|
254         hijo.hijos.each { |nieto| hijos_totales.push(nieto) }
255     end
256     hijos_totales
257 end
258
259 define_singleton_method("#{forma_de_herencia}") do |subclase|
260     super(subclase)
261     atributos = @tipos.keys
262
263     unless @listas_many.nil?
264         @listas_many.each do |atributo_lista|
265             subclase.has_many(@tipos[atributo_lista], @hashees_atributos[atributo_lista])
266         end
267         atributos = @tipos.keys.select { |atributo| !@listas_many.include?(atributo) }
268     end
269
270     unless atributos.nil?
271         atributos
272         .each { |atributo| subclase.has_one(@tipos[atributo],
@hashees_atributos[atributo]) }
273     end
274
275     @hijos.push(subclase)
276
277     nil
278 end
279
280 define_singleton_method('method_missing') do |nombre_metodo, *argumentos, &block|
281     if @tabla.respond_to?(nombre_metodo)
```

```
282         @tabla.send(nombre_metodo, *argumentos, &block)
283     else
284         super(nombre_metodo, *argumentos, &block)
285     end
286 end
287
288 define_singleton_method('respond_to_missing?') do |nombre_metodo, _include_private =
false|
289     @tabla.respond_to?(nombre_metodo) || super(nombre_metodo, false)
290 end
291
292 @tipos = {}
293
294 @tipos[descripcion[:named]] = tipo
295
296 @hashees_atributos = {}
297
298 @hashees_atributos[descripcion[:named]] = descripcion
299
300 singleton_class.attr_reader :tabla, :tipos, :hashees_atributos
301
302 define_singleton_method('crear_segun_hash') do |hash|
303     instancia = new
304
305     instancia.define_singleton_method('id') do
306         @id
307     end
308     instancia.define_singleton_method('id=') do |valor|
309         @id = valor
310     end
311
312     instancia.send(:id=, hash[:id])
313
314     @tipos.keys.each do |atributo|
315         if instancia.send(atributo).is_a?(Array) and !hash[atributo].nil?
316             ids = hash[atributo].split(',')
317             unless ids.empty?
318                 ids.each do |id|
319                     instancia.send(atributo).push(@tipos[atributo].buscar_objeto(id))
320                 end
321             end
322
323         elsif @tipos[atributo].instance_methods.include?(:save!)
324             objeto = @tipos[atributo].buscar_objeto(hash[atributo])
325             instancia.send("#{atributo}=", objeto)
326
327         else
328             instancia.send("#{atributo}=", hash[atributo])
329         end
330     end
331 end
```

```
330         end
331
332         instancia
333     end
334
335     include(Persistible)
336
337     else
338         @tipos[descripcion[:named]] = tipo
339         @hashees_atributos[descripcion[:named]] = descripcion # Podría fusionar los 2 en 1
340         agregando al hash su tipo
341     end
342
343     def has_many(tipo, descripcion)
344         if @listas_many.nil?
345             @listas_many = []
346
347             define_singleton_method('manys') do
348                 @listas_many
349             end
350         end
351
352         @listas_many.push(descripcion[:named])
353
354         has_one(tipo, descripcion)
355     end
356 end
357
```